

Aula 8 Parte 2

Avaliação de Modelos e Validação: Como Saber, com Honestidade, se um Modelo é Bom

Objetivos desta sessão

- Explicar por que o desempenho no treino **não basta** e o que significa **generalizar**.
- Distinguir os três papéis dos dados: **treino, validação e teste**.
- Aplicar e interpretar a **validação cruzada k-fold** (estratificada), reportando **média e desvio**.
- Diagnosticar **subajuste e sobreajuste** com a curva de complexidade e a curva de aprendizado.
- Escolher **métricas** adequadas ao problema, principalmente com classes desbalanceadas e erros de custos diferentes.
- Reconhecer e evitar o **vazamento de dados** usando **Pipeline**.

1. Por que avaliar? A ideia de generalização

Um modelo de aprendizado de máquina não é útil por acertar os dados que **já viu**, e sim os que **ainda não viu**. Um aluno que decora o gabarito de uma lista de exercícios acerta 100% dela e pode ir mal na prova. Da mesma forma, um modelo capaz de memorizar o treino tem erro de treino baixíssimo e, mesmo assim, pode errar muito em dados novos. A capacidade de acertar dados novos chama-se **generalização**, e o erro que o modelo cometerá neles é o **erro de generalização**.

Avaliar um modelo, portanto, é **simular o uso futuro**: separar dados que o modelo não pode ver durante o ajuste e medir o desempenho neles. Se essa separação for mal feita, a métrica deixa de representar a generalização e passa a ser apenas um número bonito.

1.1. Os três papéis dos dados

Na prática, os dados rotulados são divididos em três papéis diferentes:

Conjunto	Para que serve	Quem o usa	Quantas vezes olhar
Treino	ajustar os parâmetros do modelo (coeficientes, splits, pesos)	o algoritmo de aprendizado	quantas o algoritmo precisar
Validação	escolher hiperparâmetros e comparar modelos (k , max_depth , qual algoritmo)	você, durante o desenvolvimento	várias vezes
Teste	estimativa final e honesta do desempenho	ninguém, até o fim	uma única vez

A regra que sustenta toda a sessão: **o conjunto de teste não pode influenciar nenhuma decisão de modelagem**. No momento em que você usa o teste para escolher um hiperparâmetro, trocar de algoritmo ou decidir quais atributos manter, ele passou a ser parte do desenvolvimento e sua nota vira otimista.

2. Holdout e seus limites

O esquema mais simples é o **holdout**: separar uma parte dos dados para teste (por exemplo, 30%) e treinar com o resto — foi o que fizemos nas aulas anteriores com **train_test_split**. É rápido e fácil de entender. O problema é que o resultado depende **de quais exemplos** caíram no teste.

Para medir essa dependência, repetimos a divisão 200 vezes com o mesmo modelo (regressão logística) e o mesmo conjunto de empréstimos, mudando apenas o sorteio:

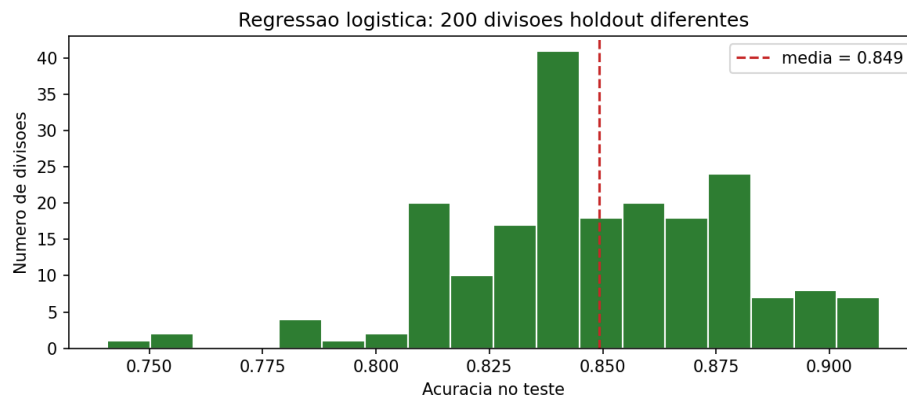


Figura 1 — distribuição da acurácia da regressão logística em 200 divisões holdout diferentes (média 0,849; desvio 0,030; mínimo 0,741; máximo 0,911).

A mesma máquina, nos mesmos dados, produziu resultados entre **74% e 91%** dependendo do sorteio. Quem tivesse feito uma única divisão poderia reportar qualquer valor dessa faixa — e nenhum deles estaria "errado". O holdout continua sendo útil, mas uma única divisão **não mostra a incerteza** da estimativa.

3. Validação cruzada k-fold

A validação cruzada resolve isso usando **todos** os exemplos, alternadamente, como treino e como validação. O procedimento com $k = 5$ é:

1. embaralhe os dados e divida-os em **5 partes** (dobras) de tamanho parecido;
2. na 1ª iteração, use a dobra 1 como **validação** e as dobras 2 a 5 como **treino**; treine e meça o desempenho;
3. repita, cada vez com uma dobra diferente como validação, até que cada dobra tenha sido validação **exatamente uma vez**;
4. reporte a **média** dos 5 desempenhos e o **desvio-padrão** entre eles.



Figura 2 — validação cruzada com 5 dobras: em cada iteração, uma dobra (vermelho) valida e as outras quatro (verde) treinam.

```
dobras = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
notas = cross_val_score(pipeline, X, y, cv=dobras)          # uma acurácia por dobra
print(f"{notas.mean():.3f} ± {notas.std():.3f}")             # média ± desvio
```

A média é uma estimativa mais estável que a de um holdout único, e o desvio diz **quanto o resultado varia** entre as dobras — informação que o holdout não fornece.

3.1. Variantes e escolhas

Variante	Como funciona	Quando usar
k-fold simples	divide em k dobras, sem olhar as classes	regressão; classes equilibradas

Variante	Como funciona	Quando usar
k-fold estratificado	cada dobra preserva a proporção de cada classe	classificação (padrão); obrigatório quando uma classe é rara
k-fold repetido	repete o k-fold várias vezes com embaralhamentos diferentes	conjuntos pequenos, para reduzir a dependência do sorteio
Leave-one-out	k = número de exemplos: cada exemplo valida uma vez	conjuntos minúsculos; é caro e a estimativa tem variância alta

Valores típicos são $k = 5$ ou $k = 10$. Mais dobras significam mais treino em cada iteração (estimativa menos pessimista), porém mais tempo de computação: a validação cruzada treina o modelo **k vezes**. O parâmetro `shuffle=True` embaralha antes de dividir, para que a ordem das linhas do arquivo não introduza viés.

4. Comparando modelos com as mesmas dobras

Para comparar algoritmos com justiça, todos devem ser avaliados nas **mesmas dobras** (o mesmo objeto `dobras`) e com o mesmo pré-processamento dentro do pipeline. Para os cinco classificadores do problema de empréstimos:

Modelo	Média	Desvio	Mínimo	Máximo
SVM (rbf)	0,860	0,038	0,800	0,900
Regressão logística	0,856	0,045	0,778	0,900
Naive Bayes	0,820	0,046	0,756	0,878
Árvore (<code>max_depth=4</code>)	0,807	0,038	0,733	0,844
k-NN (k=5)	0,798	0,035	0,756	0,856

SVM e regressão logística diferem por 0,004 — muito menos que o desvio entre as dobras (0,04). Na prática estão **empatados**. Já a distância entre o SVM (0,860) e o k-NN (0,798), de cerca de 0,06, é maior que um desvio e sugere uma diferença real.

Regra de bolso

Se a diferença entre as médias de dois modelos é **menor que a variação entre as dobras**, não declare um vencedor. Uma análise ainda melhor é **pareada**: calcular a diferença dobra a dobra e contar quantas vezes cada modelo venceu — o mini-laboratório desta sessão pratica isso.

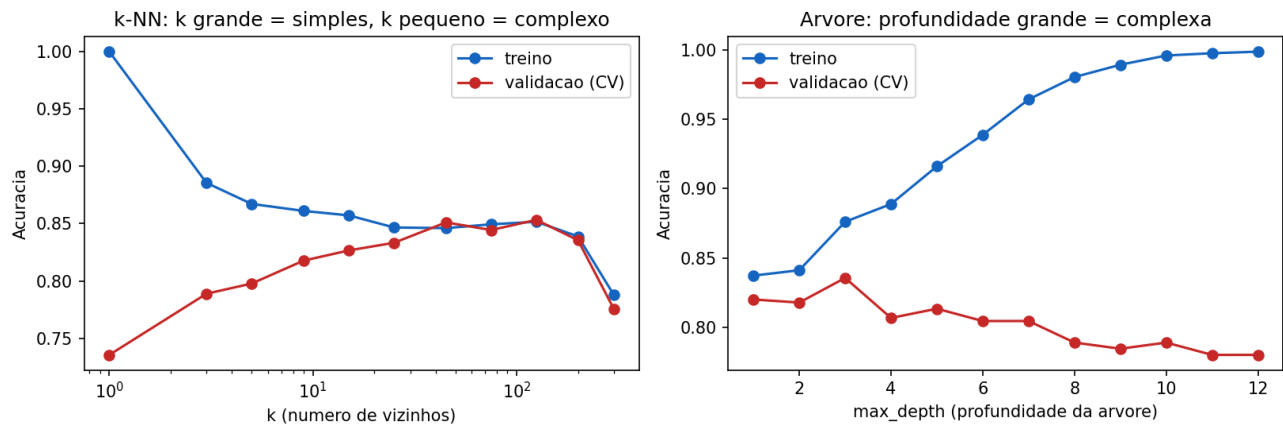
5. Subajuste e sobreajuste

Todo modelo tem um nível de **complexidade**: quanto mais flexível, mais ele consegue se ajustar aos dados. Existem dois modos clássicos de falhar:

	Subajuste (underfitting)	Bom ajuste	Sobreajuste (overfitting)
Modelo	simples demais	complexidade adequada	complexo demais
Erro no treino	alto	baixo	muito baixo
Erro na validação	alto	baixo	alto
Sintoma	erra até o que já viu; nem o treino é bem explicado	treino e validação próximos e bons	grande lacuna entre treino e validação
Exemplos	reta para dados curvos; k enorme	—	$k=1$; árvore sem limite; polinômio de grau alto

5.1. Curva de complexidade

Varia-se um **hiperparâmetro** que controla a complexidade e comparam-se o desempenho no **treino** e na **validação cruzada**. À esquerda, o k-NN (quanto **menor** o **k**, mais complexo o modelo); à direita, a árvore de decisão (quanto **maior** a profundidade, mais complexa):



Padrão observado	Diagnóstico	Curas prováveis
Grande lacuna: treino = 1,0 e validação entre 0,73 e 0,80 (árvore sem limite)	variância alta (sobreajuste)	modelo mais simples ou regularizado; mais dados
Ambas as curvas baixas e próximas	viés alto (subajuste)	modelo mais flexível; mais atributos

6. Métricas: a acurácia nem sempre conta a história

6.1. Classificação

A matriz de confusão (verdadeiros/falsos positivos e negativos) é a base das principais métricas:

```

acurácia = (VP + VN) / total      fração de acertos
precisão = VP / (VP + FP)         dos marcados como positivos, quantos eram
recall   = VP / (VP + FN)         dos positivos reais, quantos foram achados
F1       = 2 * precisão * recall / (precisão + recall)

```

A acurácia funciona bem quando as classes são equilibradas e os dois erros têm custo parecido. Quando uma classe é **rara**, ela engana. Considere um problema de detecção de fraude em que apenas 5% dos exemplos são fraude:

Modelo	Acurácia	Recall	F1	Fraudes perdidas
Baseline ("nunca há fraude")	0,950	0,000	0,000	15 de 15
Regressão logística	0,977	0,600	0,720	6 de 15
Regressão logística com <code>class_weight="balanced"</code>	0,940	0,933	0,609	1 de 15

O baseline tem 95% de acurácia **sem aprender nada** e não detecta nenhuma fraude. A regressão logística comum chega a 97,7% — parece excelente — mas deixa passar 40% das fraudes. A versão balanceada tem **acurácia menor** (até abaixo do baseline!) e, ainda assim, é a que realmente encontra as fraudes, ao custo de mais alarmes falsos. Qual é "melhor" depende do **custo de cada erro**, não da acurácia.

Métrica	Pergunta que responde	Priorize quando...
Precisão	dos que o modelo marcou como positivos, quantos eram?	o alarme falso é caro (aprovar crédito ruim, bloquear cliente honesto)
Recall	dos positivos reais, quantos o modelo achou?	perder um positivo é caro (fraude, doença grave)
F1	equilíbrio entre precisão e recall	os dois erros importam e a classe é rara

6.2. Regressão (retomada)

Na regressão as métricas medem a **distância** entre o valor previsto e o real: o **MAE** (erro absoluto médio, na unidade do alvo), o **RMSE** (raiz do erro quadrático médio, que penaliza mais os erros grandes) e o **R²** (fração da variância do alvo explicada pelo modelo; 0 equivale a sempre prever a média). Valem as mesmas regras: avalie em dados não vistos e compare com um baseline (prever a média).

7. Vazamento de dados (data leakage)

Há **vazamento** quando informação do conjunto de validação ou teste chega, de alguma forma, ao treino. O modelo então "já viu a resposta" e a métrica fica artificialmente alta — muitas vezes de forma espetacular. Fontes comuns:

- **padronizar, imputar ou selecionar atributos com o conjunto inteiro** antes de dividir em treino e validação;

- **duplicatas** ou registros quase idênticos (mesma pessoa, mesmo cliente) espalhados entre treino e teste;
- atributos que **contêm a resposta** (por exemplo, "valor já negociado" num modelo que prevê a inadimplência);
- em dados temporais, **embaralhar** e usar o futuro para prever o passado.

Para ver o tamanho do estrago, criamos 60 exemplos com 2000 atributos de **puro ruído** e alvo sorteado: não existe padrão nenhum a aprender e a acurácia honesta é 50%. Repetindo em 20 conjuntos:

Procedimento	Acurácia média
Selecionar os 20 "melhores" atributos com todos os dados e depois validar (errado)	0,915
Selecionar os atributos dentro de cada dobra, via Pipeline (correto)	0,526

O procedimento errado reporta ~91% num problema em que não há nada a aprender. A seleção de atributos olhou o alvo de **todas** as linhas, inclusive das que depois serviram de validação. O mesmo vale para imputação, padronização e qualquer passo que aprenda algo dos dados: ele deve ser **ajustado só com o treino de cada dobra**. O **Pipeline** garante isso automaticamente:

```
pipe = Pipeline([("prep", preprocessor), ("modelo", LogisticRegression())])
notas = cross_val_score(pipe, X, y, cv=dobras) # pré-processamento feito por dobra
```

8. Escolhendo hiperparâmetros sem trair o teste

Hiperparâmetros como o **k** do k-NN não são aprendidos pelo algoritmo: quem escolhe é você. O fluxo honesto usa os **três papéis**: separe o **teste** e não o toque; no restante, use **validação cruzada** para escolher; ao final, treine com a melhor configuração e meça no teste **uma única vez**. O **GridSearchCV** automatiza a busca:

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

busca = GridSearchCV(pipeline_knn,
                    param_grid={"modelo__n_neighbors": [1, 3, 5, 9, 15, 25, 45]},
                    cv=dobras)

busca.fit(X_tr, y_tr) # a validação cruzada acontece SÓ dentro do treino
print(busca.best_params_, busca.best_score_)
print(busca.score(X_te, y_te)) # teste: usado uma única vez, no final
```

k	Média na validação	Desvio	
1	0,694	0,048	
5	0,775	0,048	
15	0,825	0,052	
25	0,846	0,033	melhor k
45	0,834	0,029	

A busca escolhe **k=25** (validação cruzada 0,846) e, no teste, usado uma única vez, a acurácia é 0,867 — coerente com a estimativa. Se, em vez disso, você olhasse o teste para escolher o **k** e reportasse esse mesmo teste, a nota estaria contaminada.

9. Checklist de uma avaliação honesta

- Há um **baseline** (chute da classe mais comum, previsão da média) para comparar?
- O **teste** ficou guardado até o fim e foi usado **uma única vez**?

- Todo o **pré-processamento** está dentro do **Pipeline**, aprendido só com o treino de cada dobra?
- A comparação entre modelos usou **as mesmas dobras** e reporta **média e desvio**?
- A diferença entre dois modelos é **maior que a variação** entre as dobras?
- A **métrica** combina com o problema (classes raras, custo dos erros)?
- Foram olhadas as **curvas** (complexidade e aprendizado) para diagnosticar sub e sobreajuste?

O que guardar desta sessão

- O objetivo é **generalizar**: o desempenho no treino não diz nada sobre dados novos.
- Treino ajusta o modelo, **validação escolhe** hiperparâmetros e modelos, e o **teste** só confere no fim — uma única vez.
- Um holdout único é **instável**; a validação cruzada estratificada usa todos os dados e mostra a incerteza (média $\pm$ desvio).
- Se a diferença entre modelos é menor que a variação entre as dobras, eles estão **empatados**.
- **Subajuste** = erro alto em treino e validação; **sobreajuste** = treino ótimo e validação ruim (grande lacuna).
- **Acurácia engana** com classes desbalanceadas: use matriz de confusão, precisão, recall e F1, e pense no custo dos erros.
- **Vazamento** faz o modelo parecer mágico; o **Pipeline** evita que o pré-processamento veja dados de validação.

Arquivos desta sessão

- [lab08_parte2_demo.py](#) — demonstração guiada (8 blocos); gera as quatro figuras desta apostila.
- [1_holdout_vs_validacao_cruzada.ipynb](#), [2_subajuste_e_sobreajuste.ipynb](#) e [3_metricas_desbalanceamento_e_vazamento.ipynb](#) — notebooks com os mesmos experimentos, passo a passo.
- [lab08_parte2_exercicios.py](#) — mini-laboratório do aluno (preencha os **# TODO** e responda o RELATORIO).